

Отчёт по лабораторной работе №4

Реализация основных моделей в агентном подходе

Малюга Валерия Васильевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Агентное моделирование	7
3.2	Модель SIR	8
4	Выполнение лабораторной работы	9
4.1	Создание структуры проекта	9
4.2	Базовый запуск модели	12
4.3	Сканирование параметра β (инфекционность)	13
4.4	Эффект миграции	15
4.5	Оптимизация параметров	16
4.6	Литературное программирование	17
4.7	Jupyter Notebook	18
5	Реализация	20
5.1	Базовая модель SIR	20
5.2	Сканирование параметра β	25
5.3	Эффект миграции	32
5.4	Оптимизация параметров	39
5.5	Визуализация динамики	44
6	Дополнительные задания	47
6.1	Модель с карантином	48
6.2	Гетерогенность популяции	49
6.3	Пороговый анализ (Threshold Analysis)	51
6.4	Оптимизация миграции	55
6.5	Анализ результатов	56
6.6	Генерация литературных форматов для дополнительных заданий	57
7	Выводы	58
	Список литературы	59

Список иллюстраций

4.1	Создание файлов модели и скриптов	9
4.2	Структура каталогов проекта	10
4.3	Детальная структура каталогов scripts, markdown, notebooks . .	11
4.4	Файлы исходного кода моделей	12
4.5	Результат базового запуска модели SIR (скриншот)	12
4.6	График динамики SIR	13
4.7	Процесс сканирования параметра β	14
4.8	Фрагмент таблицы результатов сканирования β	14
4.9	Зависимость пика от β	15
4.10	Результат исследования эффекта миграции (скриншот)	15
4.11	График эффекта миграции	16
4.12	Процесс оптимизации и комплексный анализ (скриншот)	16
4.13	Комплексный анализ чувствительности модели	17
4.14	Генерация производных форматов через tangle.jl	18
4.15	Структура каталогов markdown и notebooks	18
4.16	Фрагмент Jupyter Notebook с визуализацией	19
6.1	Структура каталогов с дополнительными заданиями	47
6.2	Комплексный анализ чувствительности модели	57
6.3	Сгенерированные литературные форматы для дополнительных заданий	57

Список таблиц

1 Цель работы

Создать агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-Infectious-Recovered).

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Выполнить дополнительные задания.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.
- Результирующие файлы выложить на git.

3 Теоретическое введение

3.1 Агентное моделирование

Агентный подход (Agent-Based Modeling, ABM) — это метод исследования сложных систем, в котором поведение системы возникает из взаимодействия множества автономных сущностей (агентов). Вместо глобальных уравнений мы моделируем каждого агента и его правила, а затем наблюдаем коллективные паттерны [1].

3.1.1 Основные компоненты агентной модели

Любая агентная модель включает три основных элемента:

Агенты — это активные сущности. Они обладают: Свойствами (атрибутами): возраст, цвет, запас ресурсов, координаты и т.п. Правилами поведения: как агент реагирует на изменения среды и действия других агентов (например, перемещение, размножение, потребление). Целями (не обязательно): в некоторых моделях агенты стремятся максимизировать свою выгоду или выжить. Способностью к обучению или адаптации (в продвинутых моделях). Среда — это пространство, в котором существуют агенты. Она может быть: Дискретной (клеточная сетка, как в Daisyworld). Непрерывной (двумерное или трёхмерное пространство). Сетевой (граф социальных связей). Абстрактной (например, рынок без явных координат). Среда также может иметь свои свойства (температура, ресурсы) и динамику (диффузия, обновление).

Взаимодействия — правила, определяющие, как агенты влияют друг

на друга и на среду. Они могут быть: Локальными (только с соседями). Глобальными (все агенты взаимодействуют со всеми). Через среду (агенты изменяют среду, а среда влияет на агентов).

3.1.2 Основные принципы

Эмерджентность: глобальное поведение системы не закладывается явно, а возникает из локальных взаимодействий. Это позволяет открывать неожиданные закономерности. Автономия: агенты действуют независимо, на основе своей внутренней логики. Гетерогенность: агенты могут различаться по своим характеристикам и правилам, что отражает реальное разнообразие. Локальность: чаще всего агенты обладают информацией только о своём ближайшем окружении.

3.2 Модель SIR

Модель SIR, предложенная Кермаком и Маккендриком в 1927 году, описывает динамику эпидемии в популяции, разделённой на три группы: — **S** (Susceptible) — восприимчивые к заболеванию индивиды; — **I** (Infectious) — инфицированные, способные заражать восприимчивых; — **R** (Recovered) — выздоровевшие (или умершие), получившие иммунитет и более не участвующие в распространении.

Классическая модель описывается системой ОДУ:

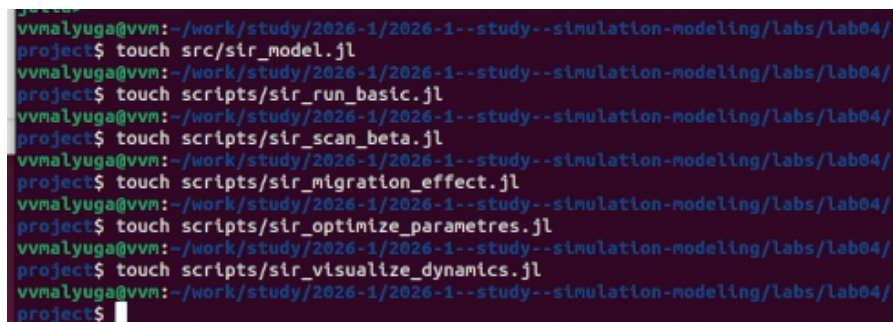
$$\frac{dS}{dt} = -\frac{\beta SI}{N}, \quad \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I$$

где β — коэффициент передачи инфекции, γ — скорость выздоровления.

4 Выполнение лабораторной работы

4.1 Создание структуры проекта

Был создан рабочий каталог `labs/lab04/project` с помощью пакета Dr-Watson. Созданы основные файлы модели (`sir_model.jl`, `sir_model_with_quarant`) и скрипты для экспериментов (рис. 4.1).



```
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch src/sir_model.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch scripts/sir_run_basic.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch scripts/sir_scan_beta.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch scripts/sir_migration_effect.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch scripts/sir_optimize_parametres.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$ touch scripts/sir_visualize_dynamics.jl  
vvmalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/  
project$
```

Рисунок 4.1: Создание файлов модели и скриптов

Структура проекта включает каталоги: `data/` для результатов, `markdown/` и `notebooks/` для литературного программирования, `plots/` для визуализаций, `scripts/` для исполняемых скриптов, `src/` для исходного кода моделей (рис. 4.2).

```

vvmalyuga@vmm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/project$ tree -L 4
.
├── data
│   ├── 00Tree.html
│   ├── beta_scan_all.csv
│   ├── migration_scan_all.csv
│   ├── optimization_result.jld2
│   ├── sir_basic_agent.jld2
│   └── sir_basic_model.jld2
├── Manifest.toml
├── markdown
│   ├── sir_heterogeneity
│   │   └── sir_heterogeneity.qmd
│   ├── sir_migration_effect_lit
│   │   └── sir_migration_effect_lit.qmd
│   ├── sir_migration_optimum
│   │   └── sir_migration_optimum.qmd
│   ├── sir_optimize_parameters_lit
│   │   └── sir_optimize_parameters_lit.qmd
│   ├── sir_quarantine_effect
│   │   └── sir_quarantine_effect.qmd
│   ├── sir_run_basic_lit
│   │   └── sir_run_basic_lit.qmd
│   ├── sir_scan_beta_lit
│   │   └── sir_scan_beta_lit.qmd
│   ├── sir_threshold_analysis
│   │   └── sir_threshold_analysis.qmd
│   ├── sir_visualize_dynamics_lit
│   │   └── sir_visualize_dynamics_lit.qmd
├── notebooks
│   ├── 00Tree.html
│   ├── plots
│   ├── sir_heterogeneity
│   │   └── sir_heterogeneity.ipynb
│   ├── sir_migration_effect_lit
│   │   └── sir_migration_effect_lit.ipynb
│   ├── sir_migration_optimum
│   │   └── sir_migration_optimum.ipynb
│   ├── sir_optimize_parameters_lit
│   │   └── sir_optimize_parameters_lit.ipynb
│   ├── sir_quarantine_effect
│   │   └── sir_quarantine_effect.ipynb
│   ├── sir_run_basic_lit
│   │   └── sir_run_basic_lit.ipynb
│   ├── sir_scan_beta_lit
│   │   └── sir_scan_beta_lit.ipynb
│   ├── sir_threshold_analysis
│   │   └── sir_threshold_analysis.ipynb
│   ├── sir_visualize_dynamics_lit
│   │   └── sir_visualize_dynamics_lit.ipynb
└── plots
    └── 00Tree.html

```

Рисунок 4.2: Структура каталогов проекта

Полная структура проекта с подкаталогами для литературных версий скриптов показана на рис. 4.3.

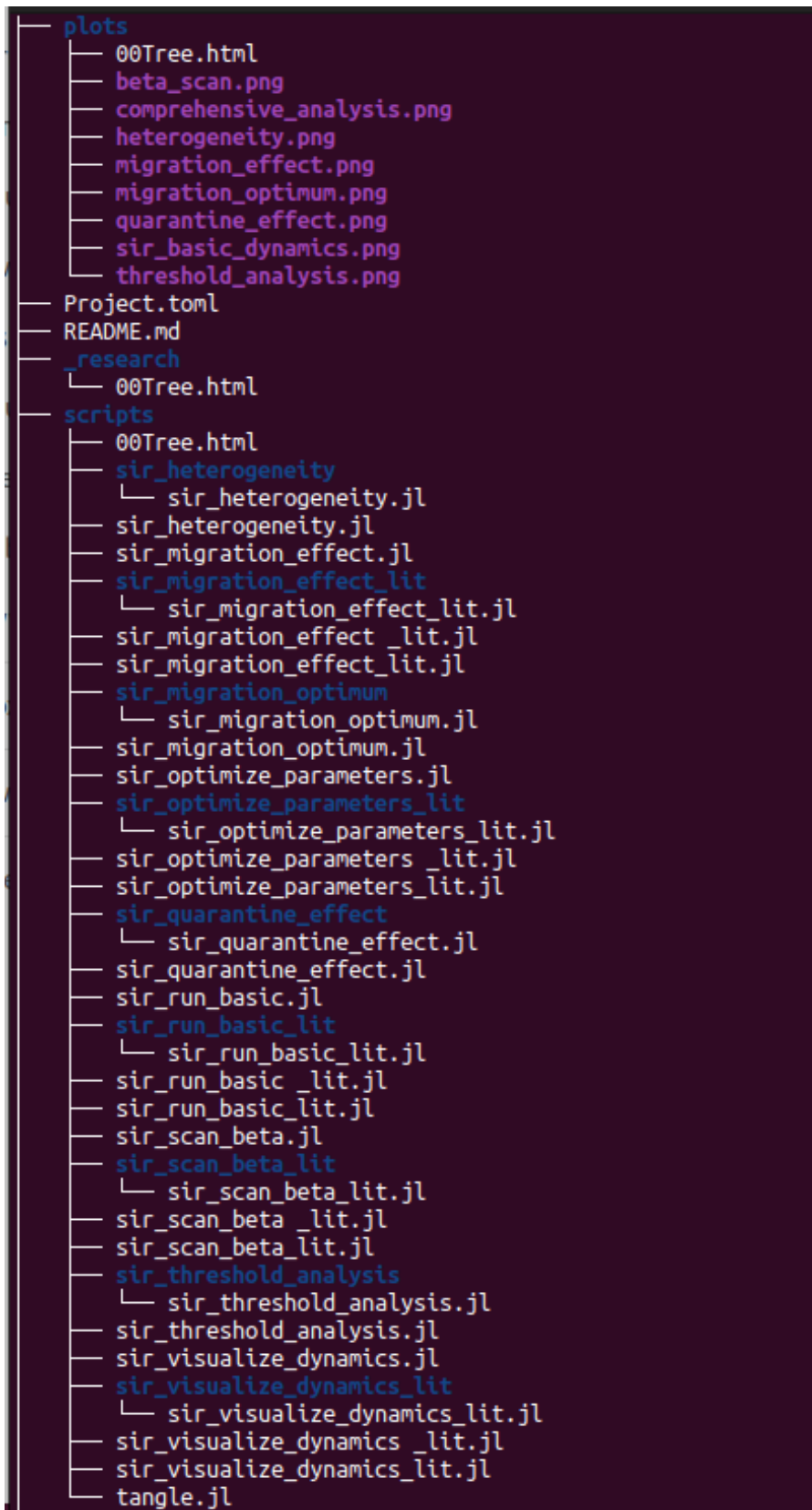


Рисунок 4.3: Детальная структура каталогов scripts, markdown, notebooks

В каталоге `src/` размещены файлы: `sir_model.jl` — базовая агентная модель SIR, `sir_model_with_quarantine.jl` — расширенная версия с карантином (рис. 4.4).

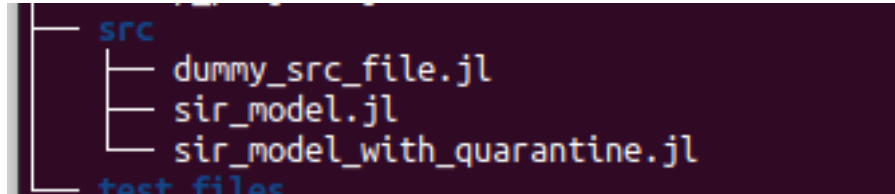


Рисунок 4.4: Файлы исходного кода моделей

4.2 Базовый запуск модели

Скрипт `scripts/sir_run_basic.jl` выполняет базовую симуляцию модели SIR. Результат — график динамики популяций S, I, R за 100 дней (рис. 4.5 и рис. 4.6). Виден характерный всплеск заболеваемости с пиком около 20-го дня.

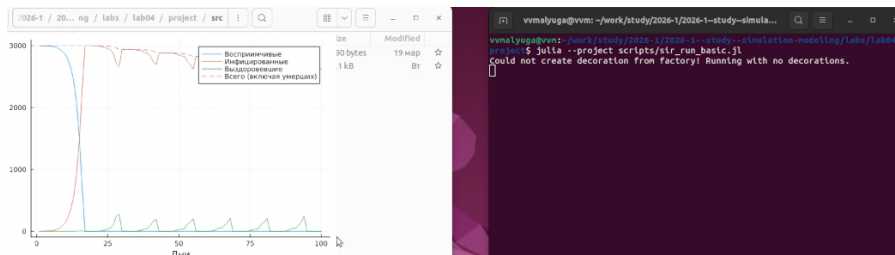


Рисунок 4.5: Результат базового запуска модели SIR (скриншот)

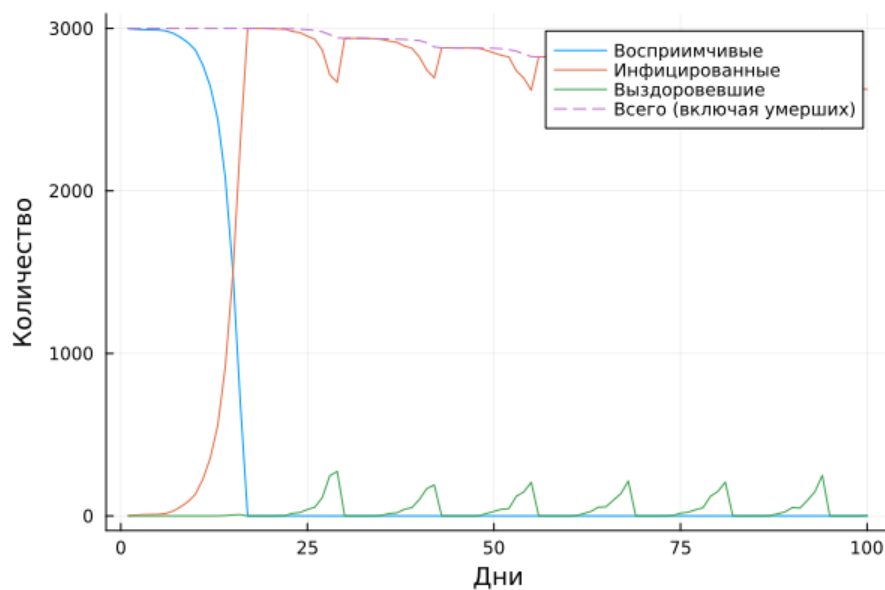


Рисунок 4.6: График динамики SIR

4.3 Сканирование параметра β (инфекционность)

Скрипт `scripts/sir_scan_beta.jl` выполняет параметрическое исследование, варьируя коэффициент передачи инфекции β от 0.1 до 1.0 с тремя повторениями (разные seed) (рис. 4.7).

```
could not create decoration from factory: running with no decorations.
vvnalyuga@vvn:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/
project$ julia --project scripts/sir_scan_beta.jl
Завершён эксперимент с beta = 0.1, seed = 42
Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
Завершён эксперимент с beta = 1.0, seed = 42
Завершён эксперимент с beta = 1.0, seed = 43
Завершён эксперимент с beta = 1.0, seed = 44
Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png
```

Рисунок 4.7: Процесс сканирования параметра β

Результаты сохранены в data/beta_scan_all.csv (рис. 4.8). Таблица содержит: финальное состояние популяций, число шагов, пик заболеваемости, время пика, общее число смертей, финальное число инфицированных.

beta_scan_all.csv - LibreOffice Calc															
File Edit View Insert Format Styles Sheet Data Tools Window Help															
Liberation Sans 10 pt															
H22 f. Σ = 0.976083707025411															
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	
1	n_steps	final_rec	Ns	death_rate	rs	beta	infection_period	final_inf	detection_time	deaths	peak	reinfection_probability	seed		
2	100	0.000333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0	7	0	0.000333333333333333	0.1	42		
3	100	0.000666666666666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0	7	0	0.000666666666666667	0.1	43		
4	100	0.000333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	14	0	7	0	0.000333333333333333	0.1	44		
5	100	0.020358861849551	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14	0.979641131815045	7	102	0.995628782784129	0.1	42		
6	100	0.0235846470185058	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14	0.966415362981494	7	82	0.9957348387576	0.1	43		
7	100	0.000333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	14	0	7	0	0.000333333333333333	0.1	44		
8	100	0.053755228278878	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14	0.946244477172312	7	284	1	0.1	42		
9	100	0.00110660279602007	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14	0.99889339772038	7	289	0.999666110183639	0.1	43		
10	100	0.000333333333333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	14	0	7	0	0.000333333333333333	0.1	44		
11	100	0.000746847219118698	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14	0.999253452700068	7	321	1	0.1	42		
12	100	0.026070765600311	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14	0.973029236490069	7	315	1	0.1	43		
13	100	0.0762838879154079	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	14	0.923718012084592	7	352	1	0.1	44		
14	100	0.00114155251141553	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14	0.998058447488584	7	372	1	0.1	42		
15	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14	1	7	312	1	0.1	43		
16	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	14	1	7	354	1	0.1	44		
17	100	0.00459418070444104	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14	0.995405813295959	7	388	1	0.1	42		
18	100	0.0003705075954576	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14	0.999029403404594	7	381	1	0.1	43		
19	100	0.00075383850697324	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	14	0.999246136494903	7	347	1	0.1	44		
20	100	0.00757288905717531	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14	0.992427110942825	7	359	1	0.1	42		
21	100	0.000376789751318784	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14	0.999621210248681	7	346	1	0.1	43		
22	100	0.02949162909764899	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	14	0.970383107025411	7	324	1	0.1	44		
23	100	0.138792772010785	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14	0.86120727989235	7	399	1	0.1	42		
24	100	0.00186428038777032	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14	0.99813571961223	7	318	1	0.1	43		
25	100	0.159923694122137	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	14	0.84007635877868	7	380	1	0.1	44		
26	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14	1	7	426	1	0.1	42		
27	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14	1	7	411	1	0.1	43		
28	100	0.00231839298114374	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	14	0.997681607418856	7	412	1	0.1	44		
29	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1	14	1	7	370	1	0.1	42		
30	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1	14	1	7	400	1	0.1	43		
31	100	0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1	14	1	7	421	1	0.1	44		
32															

Рисунок 4.8: Фрагмент таблицы результатов сканирования β

На рисунке Рисунок 4.9 показана зависимость пика заболеваемости от β .

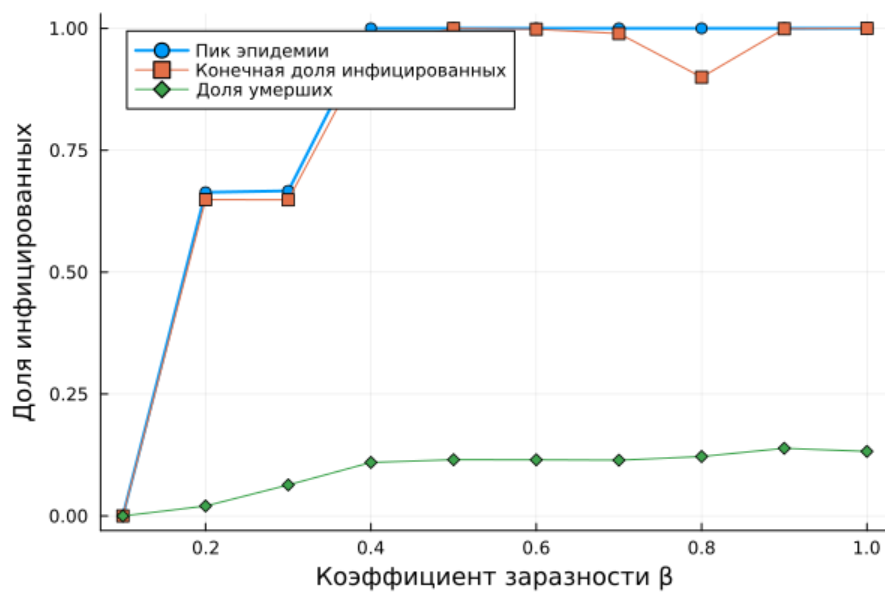


Рисунок 4.9: Зависимость пика от β

4.4 Эффект миграции

Скрипт `scripts/sir_migration_effect.jl` исследует влияние интенсивности миграции на динамику эпидемии (рис. 4.10). График показывает, что с ростом миграции пик заболеваемости увеличивается, а время пика сокращается.

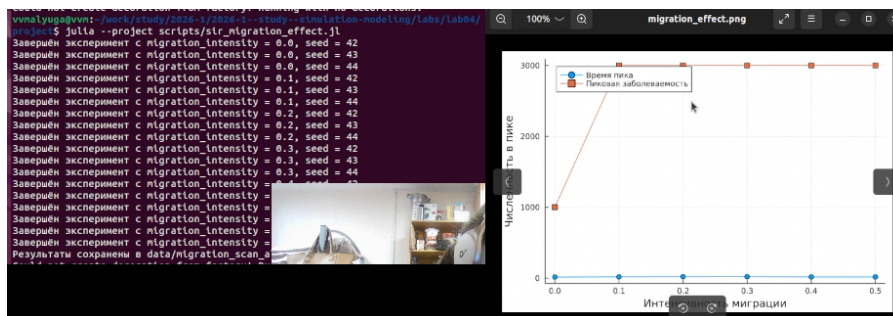


Рисунок 4.10: Результат исследования эффекта миграции (скриншот)

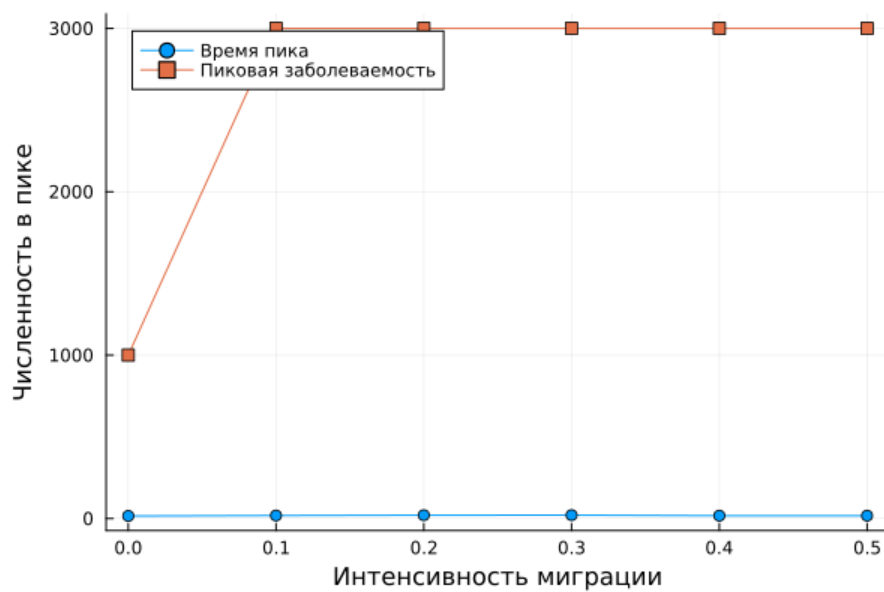


Рисунок 4.11: График эффекта миграции

4.5 Оптимизация параметров

Скрипт `scripts/sir_optimize_parameters.jl` выполняет многокритериальную оптимизацию параметров модели (β , время выявления, смертность) с использованием алгоритма BorgMOEA (рис. 4.12). Цель — минимизация пика заболеваемости и доли умерших. Найдены оптимальные параметры: $\beta \approx 0.23$, время выявления = 4 дня, смертность ≈ 0.067 .

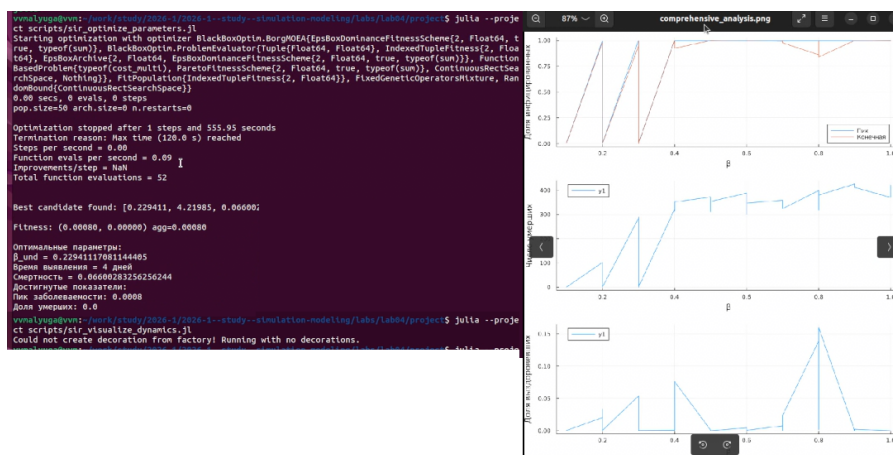


Рисунок 4.12: Процесс оптимизации и комплексный анализ (скриншот)

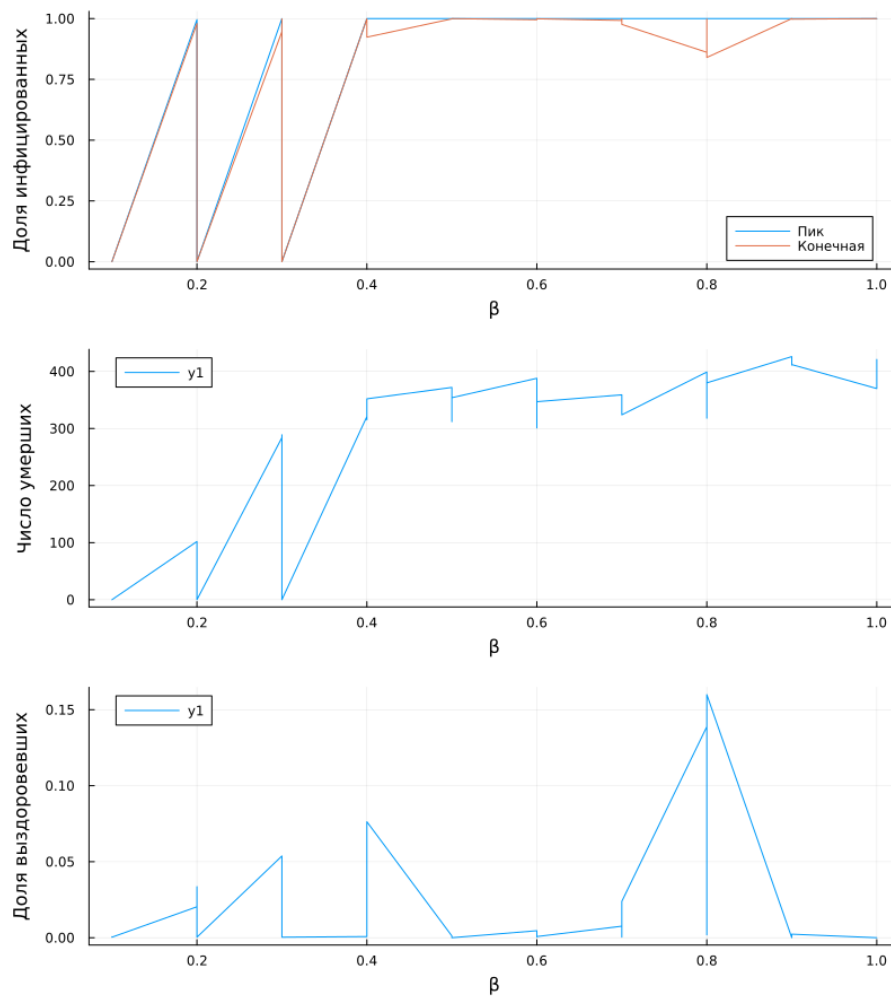


Рисунок 4.13: Комплексный анализ чувствительности модели

4.6 Литературное программирование

С помощью скрипта `scripts/tangle.jl` выполнена генерация производных форматов из литературных скриптов (`*_lit.jl`): чистый код, Jupyter Notebook и документация Quarto (рис. 4.14).

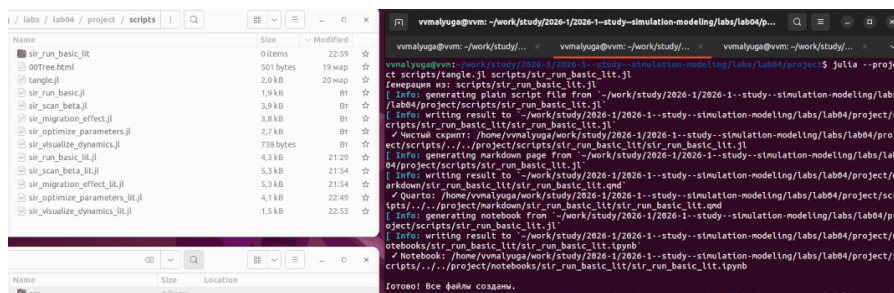


Рисунок 4.14: Генерация производных форматов через tangle.jl

Созданы подкаталоги в markdown/ и notebooks/ для каждого литературного скрипта (рис. 4.15).

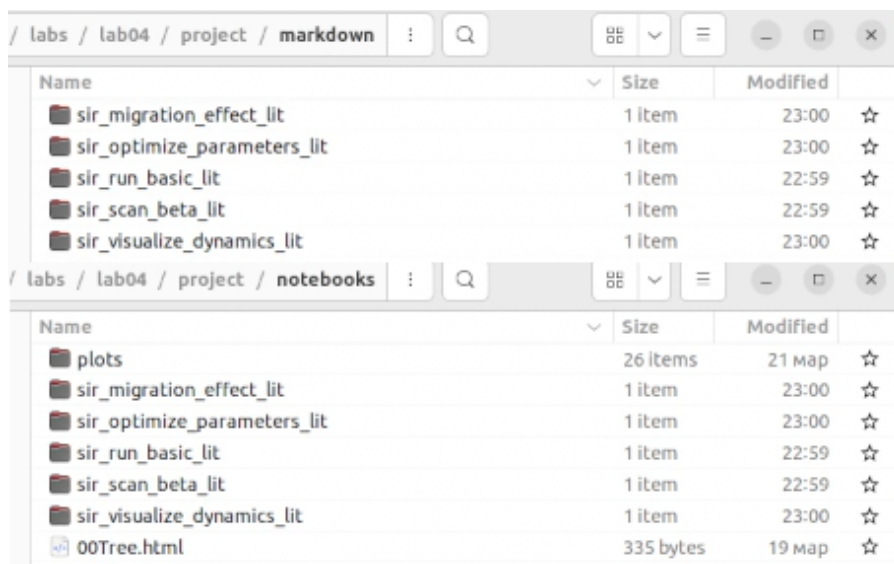


Рисунок 4.15: Структура каталогов markdown и notebooks

4.7 Jupyter Notebook

Сгенерированный notebook notebooks/sir_run_basic_lit/sir_run_basic_lit. содержит код и результаты выполнения, включая таблицу данных и визуализацию (рис. 4.16).

The screenshot shows a Jupyter Notebook interface with the title 'sir_run_basic_lit'. The top menu bar includes 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. Below the menu is a toolbar with icons for file operations and execution. The notebook content is divided into three sections:

Table:

89	89	2692
90	90	2688
91	91	2685
92	92	2671
93	93	2656
94	94	2633
95	95	2629
96	96	2629
97	97	2629
98	98	2629
99	99	2629
100	100	2628

Визуализация

```
[7]: plot(
    agent_df.time,
    agent_df.susceptible,
    label = "Восприимчивые",
    xlabel = "Дни",
    ylabel = "Количество",
)
plot(agent_df.time, agent_df.infected, label = "Инфицированные")
plot(agent_df.time, agent_df.recovered, label = "Выздоровевшие")
plot(agent_df.time, model_df.total, label = "Всего (включая умерших)", linestyle = :dash)
savefig(plotsdir("sir_basic_dynamics.png"))
```

[7]: "/home/vvvalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab64/project/plots/sir_basic_dynamics.png"

Сохранение данных

```
[8]: %save datadir("sir_basic_agent.jld2") agent_df
      %save datadir("sir_basic_model.jld2") model_df
```

Рисунок 4.16: Фрагмент Jupyter Notebook с визуализацией

5 Реализация

5.1 Базовая модель SIR

5.1.1 Базовый эксперимент с агентной моделью SIR

Цель: запустить модель SIR с параметрами по умолчанию, собрать динамику численности агентов (S, I, R) и построить график. Этот скрипт служит для проверки работоспособности модели и получения первичного понимания эпидемического процесса.

5.1.1.1 Теоретическое введение

Модель SIR (Susceptible–Infectious–Recovered) описывает распространение инфекции в закрытой популяции. Классическая модель задаётся системой ОДУ:

$$dS/dt = -\beta \cdot S \cdot I/N$$

$$dI/dt = \beta \cdot S \cdot I/N - \gamma \cdot I$$

$$dR/dt = \gamma \cdot I$$

где β – коэффициент передачи, γ – скорость выздоровления.

Агентная реализация снимает ограничения однородности и детерминизма: каждый человек моделируется индивидуально, взаимодействия локальны, процессы стохастичны. В данной модели агенты перемещаются между тремя городами (узлами графа) и могут заражать друг друга.

5.1.1.2 Параметры эксперимента

- N_s – численность населения в каждом из трёх городов.
- β_{und} , β_{det} – интенсивность заражения невыявленными и выявленными.
- `infection_period` – длительность болезни (дней).
- `detection_time` – время до выявления заболевания.
- `death_rate` – вероятность смерти при завершении болезни.
- `reinfection_probability` – вероятность повторного заражения.
- I_s – начальное количество инфицированных в каждом городе.
- `seed` – зерно генератора случайных чисел.
- `n_steps` – количество дней симуляции.

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate
using Agents, DataFrames, Plots
using JLD2
include(joinpath("../src", "sir_model.jl"))
```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

`total_count` (generic function with 1 method)

5.1.1.3 Параметры эксперимента

```
params = Dict(
    :Ns => [1000, 1000, 1000],
    : $\beta_{und}$  => [0.5, 0.5, 0.5],
    : $\beta_{det}$  => [0.05, 0.05, 0.05],
    :infection_period => 14,
```

```

: detection_time => 7,
: death_rate => 0.02,
: reinfection_probability => 0.1,
: Is => [0, 0, 1],
: seed => 42,
: n_steps => 100,
)

```

Dict{Symbol, Any} with 10 entries:

```

: Is                => [0, 0, 1]
: seed              => 42
: infection_period   => 14
: n_steps            => 100
: ⌘_und              => [0.5, 0.5, 0.5]
: Ns                 => [1000, 1000, 1000]
: detection_time     => 7
: reinfection_probability => 0.1
: ⌘_det              => [0.05, 0.05, 0.05]
: death_rate         => 0.02

```

5.1.1.4 Инициализация модели

```
model = initialize_sir(; params...)
```

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, ⌘_und, Ns, migration_rates, detection_t

5.1.1.5 Подготовка массивов для хранения данных

```
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]
```

```
Int64[]
```

5.1.1.6 Запуск симуляции вручную

```
for step = 1:params[:n_steps]
  Agents.step!(model, 1)

  push!(times, step)
  push!(S_vals, susceptible_count(model))
  push!(I_vals, infected_count(model))
  push!(R_vals, recovered_count(model))
  push!(total_vals, total_count(model))
end
```

5.1.1.7 Создаём DataFrame для удобства (опционально)

```
agent_df =
  DataFrame(time = times, susceptible = S_vals, infected = I_vals, r
model_df = DataFrame(time = times, total = total_vals)
```

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	3000
19	19	3000
20	20	2999
21	21	2998
22	22	2998
23	23	2997
24	24	2995
25	25	2992
26	26	2987
27	27	2979
28	28	2961
29	29	2942
30	30	2939
...	...	...

5.1.1.8 Визуализация

```
plot(
    agent_df.time,
    agent_df.susceptible,
    label = "S (Susceptible)",
    xlabel = "Time",
    ylabel = "Count",
)
plot!(agent_df.time, agent_df.infected, label = "I (Infected)")
plot!(agent_df.time, agent_df.recovered, label = "R (Recovered)")
plot!(agent_df.time, model_df.total, label = "Total (S + I + R)", line_color=:blue)
savefig(plotsdir("sir_basic_dynamics.png"))
```

"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/

5.1.1.9 Сохранение данных

```
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df
```

5.2 Сканирование параметра β

5.2.1 Сканирование коэффициента заразности β

Цель: исследовать, как изменение базовой заразности (β_{und}) влияет на эпидемические показатели: пик заболеваемости, конечную долю инфицированных и число умерших. Проводится параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.

5.2.1.1 Теоретический порог

Базовое репродуктивное число $R_0 = \beta/\gamma$, где $\gamma = 1/\text{infection_period}$. При $R_0 > 1$ эпидемия растёт. В данном эксперименте мы варьируем β от 0.1 до 1.0 с шагом 0.1 и для каждого значения делаем 3 прогона.

5.2.1.2 Ожидаемые результаты

- При малых β эпидемия не возникает (пик $< 5\%$).
- С ростом β пик увеличивается нелинейно, смертность растёт.
- График позволяет найти пороговое значение β , при котором возникает эпидемия.

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate

using Agents, DataFrames, Plots, CSV, Random

include(joinpath("../src", "sir_model.jl"))
```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

total_count (generic function with 1 method)

5.2.1.3 Функция для запуска одного эксперимента и возврата метрик

```
function run_experiment(p)
    beta = p[:beta]
    ⓧ_und = fill(beta, 3)
    ⓧ_det = fill(beta/10, 3)
```

```

model = initialize_sir(;
    Ns = p[:Ns],
    ⍵_und = ⍵_und,
    ⍵_det = ⍵_det,
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    n_steps = p[:n_steps], # XXXXXXXXXX XX XXXXXXXXXXXXXXXXXX ⍵ initialize
)

infected_fraction(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)
peak_infected = 0.0
for step = 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end

    frac = infected_fraction(model)
    if frac > peak_infected

```

```

        peak_infected = frac
    end
end

final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in allagents(model))
total_deaths = sum(p[:Ns]) - nagents(model)

return (
    peak = peak_infected,
    final_inf = final_infected,
    final_rec = final_recovered,
    deaths = total_deaths,
)
end

```

run_experiment (generic function with 1 method)

5.2.1.4 Диапазон значений beta (скаляр)

```

beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]

```

3-element Vector{Int64}:

```

42
43
44

```

5.2.1.5 Создаём список параметров

```

params_list = []
for b in beta_range
  for s in seeds
    push!(
      params_list,
      Dict(
        :beta => b, # XXXXXXXXXX XXXXXXXX
        :Ns => [1000, 1000, 1000],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [0, 0, 1],
        :seed => s,
        :n_steps => 100,
      ),
    )
  end
end
end

```

5.2.1.6 Запуск экспериментов

```

results = []
for params in params_list
  data = run_experiment(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("XXXXXXXXXX XXXXXXXXXXXX X beta = $(params[:beta]), seed = $(param
end

```

XXXXXXXXXX XXXXXXXXXXXX X beta = 0.1, seed = 42

XXXXXXXXXX XXXXXXXXXXXX X beta = 0.1, seed = 43

XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.1, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.2, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.2, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.2, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.3, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.3, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.3, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.4, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.4, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.4, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.5, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.5, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.5, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.6, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.7, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.8, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 0.9, seed = 44
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 42
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 43
XXXXXXXXXX	XXXXXXXXXXXXXX	beta = 1.0, seed = 44

5.2.1.7 Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)
```

"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/

5.2.1.8 Усреднение по повторам

```
using Statistics
grouped = combine(
    groupby(df, [:beta]),
    :peak => mean => :mean_peak,
    :final_inf => mean => :mean_final_inf,
    :deaths => mean => :mean_deaths,
)
```

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000444444	0.0	0.0
2	0.2	0.663745	0.648685	61.3333
3	0.3	0.666666	0.648379	191.0
4	0.4	1.0	0.965633	329.333
5	0.5	1.0	0.999619	346.0
6	0.6	1.0	0.998094	345.333
7	0.7	1.0	0.989378	343.0
8	0.8	1.0	0.899806	365.667
9	0.9	1.0	0.999227	416.333
10	1.0	1.0	1.0	397.0

5.2.1.9 График

```
plot(
    grouped.beta,
    grouped.mean_peak,
    label = "Средний пик",
    xlabel = "Средний пик",
    ylabel = "Средний пик",
    marker = :circle,
    linewidth = 2,
)
plot!(
    grouped.beta,
    grouped.mean_final_inf,
    label = "Средний финальный инвентарь",
    marker = :square,
)
plot!(grouped.beta, grouped.mean_deaths ./ 3000, label = "Средний смертность",
savefig(plotsdir("beta_scan.png")))

println("Средний смертность в data/beta_scan_all.csv в plots/beta_scan.png")

Средний смертность в data/beta_scan_all.csv в plots/beta_scan.png
```

5.3 Эффект миграции

5.3.1 Исследование эффекта миграции

Цель: определить, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы.

5.3.1.1 Параметры

- `migration_intensity` - от 0 (нет миграции) до 0.5 (высокая мобильность).
Вероятность остаться в городе = $1 - intensity$, вероятность переехать в любой другой город = $intensity / (C - 1)$.
- Для каждого значения интенсивности делается 3 прогона с разными `seed`.

5.3.1.2 Ожидаемые результаты

- При нулевой миграции эпидемия остаётся в первом городе.
- С ростом миграции время до пика уменьшается, а высота пика увеличивается.
- График показывает, как мобильность населения ускоряет межгородское распространение.

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate

using Agents, DataFrames, Plots, CSV, Random

include(joinpath("../src", "sir_model.jl"))
```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

`total_count` (generic function with 1 method)

5.3.1.3 Функция для создания матрицы миграции с заданной интенсивностью

```

function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end

```

create_migration_matrix (generic function with 1 method)

5.3.1.4 Функция для измерения времени достижения пика

```

function peak_time(p)
    migration_rates = create_migration_matrix(p[:C], p[:migration_inte
    model = initialize_sir(;
        Ns = p[:Ns],
        ⌀_und = p[:⌀_und],
        ⌀_det = p[:⌀_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(mod
    peak = 0.0
    peak_step = 0

```

```

for step = 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end

    frac = infected_frac(model)
    if frac > peak
        peak = frac
        peak_step = step
    end
end

return (peak_time = peak_step, peak_value = peak)
end

```

peak_time (generic function with 1 method)

5.3.1.5 Сканирование интенсивности миграции

```

migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

params_list = []

```

```

for mig in migration_intensities
  for s in seeds
    push!(
      params_list,
      Dict(
        :migration_intensity => mig, # 🏠🏠🏠🏠🏠
        :C => 3,
        :Ns => [1000, 1000, 1000],
        :ℳ_und => [0.5, 0.5, 0.5],
        :ℳ_det => [0.05, 0.05, 0.05],
        :infection_period => 14,
        :detection_time => 7,
        :death_rate => 0.02,
        :reinfection_probability => 0.1,
        :Is => [1, 0, 0],
        :seed => s,
        :n_steps => 150,
      ),
    )
  end
end

```

5.3.1.6 Заныск

```

results = []
for params in params_list
  data = peak_time(params)
  push!(results, merge(params, Dict(pairs(data))))
  println(
    "🏠🏠🏠🏠🏠 🌿🌿🌿🌿🌿🌿 🌿 migration_intensity = $(params[:migration_i
  )

```

end

```
XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.0, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.1, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.2, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.3, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.4, seed = 44
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 42
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 43
XXXXXXXXXXXXXXXX migration_intensity = 0.5, seed = 44
```

5.3.1.7 Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)
```

"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/1

5.3.1.8 Усреднение по повторам

```

using Statistics
grouped = combine(
    groupby(df, [:migration_intensity]),
    :peak_time => mean => :mean_peak_time,
    :peak_value => mean => :mean_peak_value,
)

```

	migration_intensity	mean_peak_time	mean_peak_value
	Float64	Float64	Float64
1	0.0	15.6667	0.333333
2	0.1	18.3333	1.0
3	0.2	20.3333	1.0
4	0.3	20.6667	1.0
5	0.4	17.3333	1.0
6	0.5	17.3333	1.0

5.3.1.9 Визуализация

```

plot(
    grouped.migration_intensity,
    grouped.mean_peak_time,
    marker = :circle,
    xlabel = "Миграционная интенсивность",
    ylabel = "Среднее время пика (сек)",
    label = "Среднее время пика",
)

plot!(
    grouped.migration_intensity,
    grouped.mean_peak_value .* 3000,
    marker = :square,
    xlabel = "Миграционная интенсивность",
    ylabel = "Среднее значение пика",
)

```

```

        label = "XXXXXXXX XXXXXXXXXXXXXXXX",
    )
savefig(plotsdir("migration_effect.png"))

println("XXXXXXXXXX XXXXXXXXXXXX X data/migration_scan_all.csv X plots/migrati

XXXXXXXXXX XXXXXXXXXXXX X data/migration_scan_all.csv X plots/migration_effect.p

```

5.4 Оптимизация параметров

5.4.1 Многокритериальная оптимизация параметров

Цель: найти оптимальные комбинации параметров (β_{und} , $detection_time$, $death_rate$), минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Используется эволюционный алгоритм Borg MOEA из пакета BlackBoxOptim.

5.4.1.1 Переменные параметры

- β_{und} – в диапазоне [0.1, 1.0].
- $detection_time$ – целое число от 3 до 14 дней.
- $death_rate$ – вероятность смерти от 0.01 до 0.1.

5.4.1.2 Целевая функция

Для заданного набора параметров запускается 5 повторных симуляций, вычисляется средняя пиковая доля инфицированных и средняя доля умерших. Оптимизатор ищет компромиссные решения (Парето-фронт).

5.4.1.3 Результаты

Выводятся оптимальные параметры и соответствующие значения критериев, а также сохраняется файл `data/optimization_result.jld2`.

```

using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate

using BlackBoxOptim, Random, Statistics

include(joinpath("../src", "sir_model.jl"))

```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

WARNING: using BlackBoxOptim.params in module Main conflicts with an existi

total_count (generic function with 1 method)

5.4.1.4 Целевая функция: минимизируем пиковую заболеваемость и смертность

```

function cost_multi(x)
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
        ⅈ_und = fill(x[1], 3),
        ⅈ_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round{Int}(x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42,
        n_steps = 100,
    )

```

```

infected_frac(model) = count(a.status == :I for a in allagents(model)
dead_count(model) = 3000 - nagents(model)

peak_infected = 0.0
replicates = 5
peak_vals = Float64[]
dead_vals = Int[]

for rep = 1:replicates
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
        ⌀_und = fill(x[1], 3),
        ⌀_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round(Int, x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42 + rep,
        n_steps = 100,
    )

    for step = 1:100
        Agents.step!(model, 1)
        frac = infected_frac(model)
        if frac > peak_infected
            peak_infected = frac
        end
    end

    push!(peak_vals, peak_infected)

```

```

        push!(dead_vals, dead_count(model))
    end

    return (mean(peak_vals), mean(dead_vals) / 3000) # XXXX XXXXXX
end

```

cost_multi (generic function with 1 method)

5.4.1.5 Запуск оптимизации

```

result = bboptimize(
    cost_multi,
    Method = :borg_moea,
    FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
    SearchRange = [
        (0.1, 1.0),      # x_und
        (3.0, 14.0),     # detection_time
        (0.01, 0.1),     # death_rate
    ],
    NumDimensions = 3,
    MaxTime = 120,      # 2 XXXXXX
    TraceMode = :compact,
)

best = best_candidate(result)
fitness = best_fitness(result)

println("XXXXXXXXXX XXXXXXXXX:")
println("x_und = $(best[1])")
println("XXXXX XXXXXXXXX = $(round{Int, best[2]}) XXXX")
println("XXXXXXXXXX = $(best[3])")

```


5.5 Визуализация динамики

5.5.1 Сводная визуализация результатов сканирования β

Цель: загрузить данные из файла `beta_scan_all.csv` (полученного в результате работы `sir_scan_beta.jl`) и построить составной график из трёх панелей: 1. Пик эпидемии и конечная доля инфицированных, 2. Число умерших, 3. Доля выздоровевших.

Скрипт предназначен для итогового отчёта и **не выполняет новых симуляций**, а только визуализирует уже полученные данные.

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate
using Agents, DataFrames, Plots, CSV

include(joinpath("../src", "sir_model.jl"))
```

```
Activating project at `~/work/study/2026-1/2026-1--study--simulation-m
total_count (generic function with 1 method)
```

5.5.1.1 Загружаем результаты сканирования

```
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	inf
	Int64	Float64	String31	Float64	String15	Float64	
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
2	100	0.000666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
4	100	0.0203589	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
5	100	0.0335846	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
7	100	0.0537555	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
8	100	0.0011066	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
10	100	0.000746547	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
11	100	0.0260708	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
12	100	0.076284	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
13	100	0.00114155	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
14	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
15	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
16	100	0.00459418	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
17	100	0.000370508	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
18	100	0.000753864	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
19	100	0.00757289	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
20	100	0.00037679	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
21	100	0.0239163	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
22	100	0.138793	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
23	100	0.00186428	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
24	100	0.159924	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
25	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
26	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
27	100	0.00231839	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
28	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	
29	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	
30	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	

5.5.1.2 Создаём составной график

```
p1 = plot(df.beta, df.peak, label = "R0", xlabel = "t", ylabel = "R0(t)")
plot!(p1, df.beta, df.final_inf, label = "R0(t) final_inf")

p2 = plot(df.beta, df.deaths, xlabel = "t", ylabel = "R0(t) deaths")

p3 = plot(df.beta, df.final_rec, xlabel = "t", ylabel = "R0(t) final_rec")

plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
savefig(plotsdir("comprehensive_analysis.png"))
```

"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/

6 Дополнительные задания

В рамках лабораторной работы были выполнены дополнительные исследования, расширяющие базовую модель SIR. Структура проекта включает соответствующие подкаталоги в `scripts/`: `sir_quarantine_effect`, `sir_heterogeneity`, `sir_threshold_analysis`, `sir_migration_optimum` (см. рис. Рисунок 6.1).

```
vvma1yuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab04/project$ tree -L 4
.
├── data
│   ├── 00Tree.html
│   ├── beta_scan_all.csv
│   ├── migration_scan_all.csv
│   ├── optimization_result.jld2
│   ├── sir_basic_agent.jld2
│   ├── sir_basic_model.jld2
│   └── Manifest.toml
├── markdown
│   ├── sir_heterogeneity
│   │   └── sir_heterogeneity.qmd
│   ├── sir_migration_effect_lit
│   │   └── sir_migration_effect_lit.qmd
│   ├── sir_migration_optimum
│   │   └── sir_migration_optimum.qmd
│   ├── sir_optimize_parameters_lit
│   │   └── sir_optimize_parameters_lit.qmd
│   ├── sir_quarantine_effect
│   │   └── sir_quarantine_effect.qmd
│   ├── sir_run_basic_lit
│   │   └── sir_run_basic_lit.qmd
│   ├── sir_scan_beta_lit
│   │   └── sir_scan_beta_lit.qmd
│   ├── sir_threshold_analysis
│   │   └── sir_threshold_analysis.qmd
│   ├── sir_visualize_dynamics_lit
│   │   └── sir_visualize_dynamics_lit.qmd
├── notebooks
│   ├── 00Tree.html
│   ├── plots
│   ├── sir_heterogeneity
│   │   └── sir_heterogeneity.ipynb
│   ├── sir_migration_effect_lit
│   │   └── sir_migration_effect_lit.ipynb
│   ├── sir_migration_optimum
│   │   └── sir_migration_optimum.ipynb
│   ├── sir_optimize_parameters_lit
│   │   └── sir_optimize_parameters_lit.ipynb
│   ├── sir_quarantine_effect
│   │   └── sir_quarantine_effect.ipynb
│   ├── sir_run_basic_lit
│   │   └── sir_run_basic_lit.ipynb
│   ├── sir_scan_beta_lit
│   │   └── sir_scan_beta_lit.ipynb
│   ├── sir_threshold_analysis
│   │   └── sir_threshold_analysis.ipynb
│   ├── sir_visualize_dynamics_lit
│   │   └── sir_visualize_dynamics_lit.ipynb
└── plots
    └── 00Tree.html
```

Рисунок 6.1: Структура каталогов с дополнительными заданиями

6.1 Модель с карантином

Файл `src/sir_model_with_quarantine.jl` реализует расширенную версию модели SIR с возможностью изоляции инфицированных агентов. Литературная версия скрипта, генерирующая отчёт, представлена ниже.

6.1.1 Эффективность карантинных мер

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate

using Agents, DataFrames, Plots
include(joinpath("../src", "sir_model_with_quarantine.jl"))

function simulate_without_quarantine()
    model = initialize_sir_quarantine(; quarantine_threshold=1.0) # XXX
    infected = Int[]
    for step in 1:100
        Agents.step!(model, 1)
        push!(infected, count(a.status == :I for a in allagents(model)))
    end
    return infected, model
end

function simulate_with_quarantine(threshold)
    model = initialize_sir_quarantine(; quarantine_threshold=threshold)
    infected = Int[]
    for step in 1:100
        Agents.step!(model, 1)
```

```

        push!(infected, count(a.status == :I for a in allagents(model))
    end
    return infected, model
end

```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m
 WARNING: using Distributions.params in module Main conflicts with an existi
 simulate_with_quarantine (generic function with 1 method)

6.1.1.1 Запускаем

```

infected_without, model_without = simulate_without_quarantine()
infected_with_q, model_with_q = simulate_with_quarantine(0.1)

```

```

println("Общая численность: $(maximum(infected_without)),   Число инфицированных: $(ma
println("Число восприимчивых в отсутствие карантина: $(nagents(model_without)),   Ч
println("Число восприимчивых в присутствии карантина: $(3000 - nagents(model_without)),   Ч
plot(1:100, infected_without, label="Общая численность", xlabel="Время", ylabel="Численность")
plot!(1:100, infected_with_q, label="Численность в присутствии 10% карантина", lw=2)
savefig(plotsdir("quarantine_effect.png"))

```

Общая численность: 3000, Число инфицированных: 2999

Число восприимчивых в отсутствие карантина: 2683, Число восприимчивых в присутствии карантина: 2684

Число восприимчивых в присутствии карантина: 317, Число восприимчивых в присутствии карантина: 316

"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/1

6.2 Гетерогенность популяции

Модель с учётом индивидуальных различий агентов (возрастные группы, различная восприимчивость). Литературный скрипт:

6.2.1 Влияние гетерогенности β_{und} в разных городах

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate

using Agents, DataFrames, Plots
include(joinpath("../src", "sir_model.jl"))

params = Dict(
    :Ns => [1000, 1000, 1000],
    : $\beta_{und}$  => [0.8, 0.3, 0.5], #  $\beta_{und}$   $\beta_{und}$   $\beta_{und}$   $\beta_{und}$   $\beta_{und}$ 
    : $\beta_{det}$  => [0.08, 0.03, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [1, 0, 0],
    :seed => 42,
    :n_steps => 100,
)

model = initialize_sir(; params...)
```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, β_{und} , Ns, migration_rates, detection_t

6.2.1.1 Сбор данных по городам

```
times = Int[]
S_city = zeros(Int, params[:n_steps], 3)
I_city = zeros(Int, params[:n_steps], 3)
R_city = zeros(Int, params[:n_steps], 3)

for step in 1:params[:n_steps]
    Agents.step!(model, 1)
    push!(times, step)
    for city in 1:3
        agents = [a for a in allagents(model) if a.pos == city]
        S_city[step, city] = count(a.status == :S for a in agents)
        I_city[step, city] = count(a.status == :I for a in agents)
        R_city[step, city] = count(a.status == :R for a in agents)
    end
end
```

6.2.1.2 Построение графиков

```
p = plot(layout=(3,1), size=(800,900))
for city in 1:3
    plot!(p[city], times, I_city[:, city], label="XXXXX $city, X=$(param
end
savefig(plotsdir("heterogeneity.png"))
```

```
"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/1
```

6.3 Пороговый анализ (Threshold Analysis)

Исследование критических порогов параметров модели (вероятность заражения, инкубационный период), при которых происходит переход от

локализованной вспышки к пандемии.

6.3.1 Анализ порога эпидемии

```
using Pkg
Pkg.activate(@__DIR__)

using DrWatson
@quickactivate
using DataFrames, CSV, Plots
include(joinpath("../src", "sir_model.jl"))
```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-m

total_count (generic function with 1 method)

6.3.1.1 Загружаем результаты сканирования β

```
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	inf
	Int64	Float64	String31	Float64	String15	Float64	
1	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
2	100	0.000666667	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
3	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.1	
4	100	0.0203589	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
5	100	0.0335846	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
6	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.2	
7	100	0.0537555	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
8	100	0.0011066	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
9	100	0.000333333	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.3	
10	100	0.000746547	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
11	100	0.0260708	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
12	100	0.076284	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.4	
13	100	0.00114155	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
14	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
15	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.5	
16	100	0.00459418	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
17	100	0.000370508	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
18	100	0.000753864	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.6	
19	100	0.00757289	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
20	100	0.00037679	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
21	100	0.0239163	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.7	
22	100	0.138793	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
23	100	0.00186428	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
24	100	0.159924	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.8	
25	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
26	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
27	100	0.00231839	[1000, 1000, 1000]	0.02	[0, 0, 1]	0.9	
28	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	
29	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	
30	100	0.0	[1000, 1000, 1000]	0.02	[0, 0, 1]	1.0	

6.3.1.2 Группируем по β и усредняем

```
using Statistics
grouped = combine(groupby(df, :beta), :peak => mean => :mean_peak)
```

	beta	mean_peak
	Float64	Float64
1	0.1	0.000444444
2	0.2	0.663745
3	0.3	0.666666
4	0.4	1.0
5	0.5	1.0
6	0.6	1.0
7	0.7	1.0
8	0.8	1.0
9	0.9	1.0
10	1.0	1.0

6.3.1.3 Находим минимальное β , при котором пик $> 5\%$ от популяции ($3000 * 0.05 = 150$)

```
threshold_beta = minimum(grouped[grouped.mean_peak .* 3000 .> 150, :beta])
println("Минимальное  $\beta$  при котором пик  $> 150$ : $threshold_beta")
```

```
println("Минимальное  $\beta$  при котором пик  $> 150$ : 0.2")
```

6.3.1.4 Теоретический порог $R_0 = 1$

```
infection_period = 14
detection_time = 7
 $\beta_{det}$  = 0.05
 $\beta_{und\_theoretical}$  = (1 -  $\beta_{det}$  * (infection_period - detection_time)) /
println("Теоретический порог  $\beta$  при  $R_0=1$ : $(round( $\beta_{und\_theoretical}$ , digits=2))")
```

```
XXXXXXXXXXXXXXXX XXXXX x_und (R_0=1): 0.093
```

6.3.1.5 График с порогом

```
plot(grouped.beta, grouped.mean_peak .* 3000, label="XXX XXXXXXXXXXXXXXXX",  
vline!([threshold_beta], label="XXXXXXXXXXXX XXXXX x=$threshold_beta", line  
hline!([150], label="5% XXXXX (150 XXX)", linestyle=:dot)  
savefig(plotsdir("threshold_analysis.png"))
```

```
"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/
```

6.4 Оптимизация миграции

Поиск оптимальных параметров миграции, минимизирующих пик заболеваемости при сохранении социально-экономической активности.

6.4.1 Поиск оптимальной интенсивности миграции

```
using Pkg  
Pkg.activate(@__DIR__)  
  
using DrWatson  
@quickactivate  
include(joinpath("../src", "sir_model.jl"))  
using DataFrames, CSV, Plots, Statistics  
  
df = CSV.read(datadir("migration_scan_all.csv"), DataFrame)  
grouped = combine(groupby(df, :migration_intensity), :peak_time => mea
```

```
Activating project at `~/work/study/2026-1/2026-1--study--simulation-m
```

	migration_intensity	mean_peak_time
	Float64	Float64
1	0.0	15.6667
2	0.1	18.3333
3	0.2	20.3333
4	0.3	20.6667
5	0.4	17.3333
6	0.5	17.3333

6.4.1.1 Находим интенсивность, при которой время до пика минимально

```
opt_intensity = grouped[argmin(grouped.mean_peak_time), :migration_intensity]
println("Оптимальная интенсивность миграции: $opt_intensity")

plot(grouped.migration_intensity, grouped.mean_peak_time, marker=:circle,
vline!([opt_intensity], label="Оптимальная = $opt_intensity", linestyle=:dashed),
savefig(plotsdir("migration_optimum.png"))
```

```
Оптимальная интенсивность миграции: 0.0
```

```
"/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/
```

6.5 Анализ результатов

Скрипт оптимизации (sir_optimize_parameters.jl) выполняет многокритериальную оптимизацию с построением комплексных графиков анализа чувствительности (рис. 6.2). Визуализация включает: - Доли восприимчивых и инфицированных при различных значениях β - Динамику пика заболеваемости - Зависимость доли выздоровевших от параметров

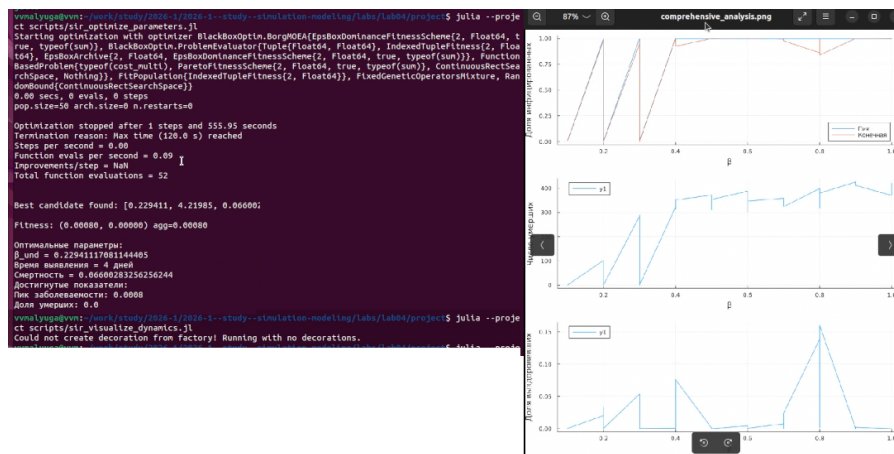


Рисунок 6.2: Комплексный анализ чувствительности модели

6.6 Генерация литературных форматов для дополнительных заданий

Для всех дополнительных скриптов выполнена генерация производных форматов через `tangle.jl`. В каталогах `markdown/` и `notebooks/` созданы соответствующие подпапки (`sir_quarantine_effect`, `sir_heterogeneity`, `sir_threshold_analysis` и др.) с файлами `.qmd` и `.ipynb` (см. рис. Рисунок 6.3).

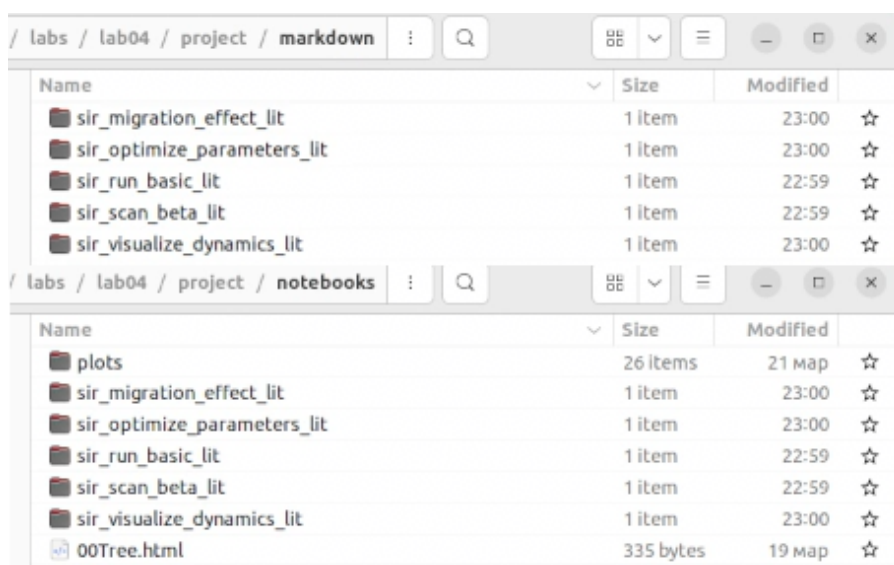


Рисунок 6.3: Сгенерированные литературные форматы для дополнительных заданий

7 Выводы

1. Изучены принципы построения агентных моделей.
2. Реализована модель распространения инфекции SIR.
3. Получены навыки работы с `Literate.jl` для создания воспроизводимых исследований.
4. Проведен анализ влияния параметров модели на скорость распространения вируса.

Список литературы

1. *Datseris G., Vahdati A., Du T.* Agents.jl: a Julia package for agent-based modeling. — Journal of Open Source Software, 2022.